

UGTS-KC 3.6.3 SARA

Defensive Red-Team Security Audit

Does it work, can it be misused, and can it actually hack a wallet?

BOTTOM LINE

The standards-conformance demo works. The public certificate is secret-free. But the claimed authorization boundary is a policy sign, not a locked door, and the pure-Python secret-key path is not safe for production wallet use.

Prepared for Tom Klootwijk

Audit date: 18 August 2026 | Europe/Amsterdam

Scope: local package only; no third-party wallet targeting, key search, transaction signing, broadcasting, or funds access

1. Executive answer

ELI5 verdict: the calculator works; the security fence does not.

The package correctly performs BIP39, BIP32, BIP84, Bech32 and Bech32m public test-vector work. It cannot turn a public address back into a wallet key. However, any Python caller can skip the separate authorization function and call the secret-derivation functions directly. That makes the gate advisory, not enforced.

Question	Verdict	Plain-English meaning
Does the reference demo actually work?	PASS	Yes. The final isolated run passed 195/195 tests in 2.097 seconds; 42 are SARA-focused.
Does it match public Bitcoin standards vectors?	PASS	Yes for the covered vectors: BIP39 seed, three BIP84 addresses, Bech32 v0, and Bech32m v1.
Can a public Bitcoin address reveal its private key?	NO	No practical inverse route was found or implemented. Address decoding returns public script facts only.
Can it brute-force a random 12-word mnemonic?	NO	Not realistically. At this laptop's measured full-derivation rate, expected work is about 1.49e30 years.
Can a stolen mnemonic plus weak passphrase be attacked?	YES	Yes in principle. A one-million-item dictionary is about 38.3 hours expected on this unoptimized Python implementation.
Is it production wallet code?	FAIL	No. Secret objects are loggable, memory is not wiped, and elliptic-curve math is variable-time.
Is the authorization gate a real access-control boundary?	FAIL	No. Direct exported APIs do not require or receive an authorization certificate.
Is the current extracted copy integrity-clean?	FAIL	No. 150/152 manifest entries match; README.md and SECURITY.md differ from the pristine ZIP.

Overall classification

Standards demo	Policy enforcement	Production wallet	Wallet hacking
PASS	FAIL	NO	NO
Useful defensive reference	Gate is bypassable	Use audited constant-time tooling	No search, signing, or network

2. Scope and method

This was a local, non-destructive red-team review. The word 'red-team' here means trying to break the package's claims and controls using only the supplied code, public test vectors, synthetic inputs, and this laptop. It did not mean attacking a real wallet.

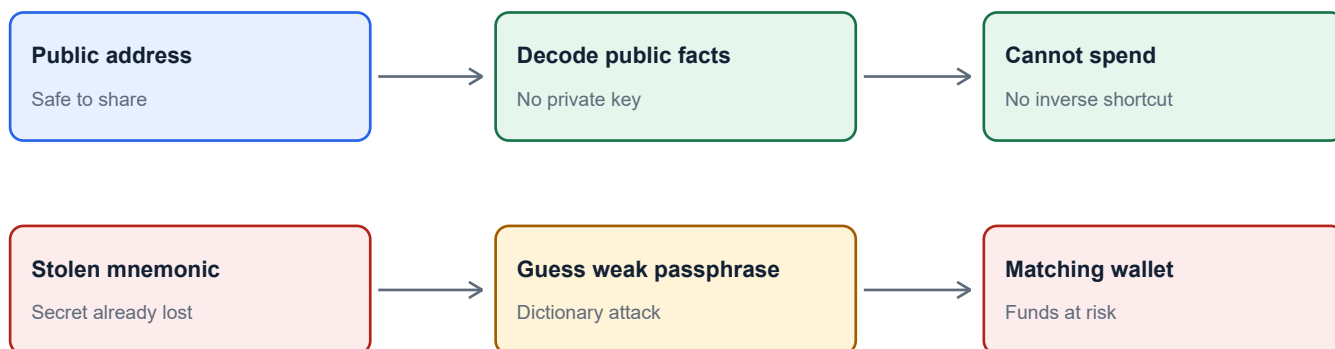
In scope

- Clean Python setup and reproducibility.
- Pristine ZIP and extracted-file integrity.
- Official public standards test vectors and package tests.
- Source, dependency, package, secret, parser, runtime, and permission review.
- Non-destructive bypass, timing, malformed-input, and local denial-of-service probes.

Explicitly out of scope

- No mnemonic enumeration, passphrase spray against a real wallet, private-key search, balance targeting, transaction signing, broadcast, or funds movement.
- No live mainnet queries and no attempt to prove ownership of the supplied public address.
- No formal cryptographic proof, exhaustive fuzzing, malware analysis, physical memory acquisition, or laboratory side-channel attack.

Two very different paths



What an address is

Think of a Bitcoin address as a letter-box slot. It tells people where a payment can go. It is not the key that opens the box. The package can inspect that slot; it does not discover the private key behind it.

3. Setup and reproducibility

Final tests	SARA-focused	Branch coverage	Demo runtime
195/195	42	86%	0.361 s
2.097 seconds	standards and boundary tests	SARA module: 86%	valid public certificate

Test machine

Question	Verdict	Plain-English meaning
Operating system	Windows 11 Home	Version 10.0.26200, build 26200.
Processor	Intel Core Ultra 7 255HX	20 logical processors.
Memory	15.42 GiB	Reported visible system memory.
Python	3.12.13	Isolated .venv-security environment.
pip	26.2.1	Updated before final dependency audit.

What failed before the green result

- An editable install with --no-build-isolation initially failed because the clean venv had no local setuptools backend.
- The first test discovery then produced four import errors because jsonschema was required by tests but not declared in project metadata.
- The validator also requires PyMuPDF through the deprecated fitz import, but that validator dependency is undeclared.
- After installing and pinning the missing audit/test tools locally, disabling inherited system packages, and rerunning, all 195 tests passed.

Why this matters

A source-tree test can be correct and still be awkward to reproduce. The final pass is real, but another user needs the missing test/validator dependency instructions or metadata to reproduce it cleanly.

4. Standards conformance: what passed

The strongest positive evidence is not that the package says 'valid'; it is that the code reproduced published public vectors and rejected known invalid encodings.

ID	Risk	Finding	Why it matters
C-01	POSITIVE	BIP39 public vector passes	The official 12-word zero-entropy fixture, checksum geometry, 2,048-word list fingerprint, NFKD handling, PBKDF2-HMAC-SHA512 seed with 2,048 iterations, and TREZOR vector all pass.
C-02	POSITIVE	BIP84 public vectors pass	The first receiving, second receiving, and first change addresses and compressed public keys match BIP84's published vectors.
C-03	POSITIVE	Bech32 and Bech32m pass	Covered v0 P2WPKH and v1 32-byte program vectors pass; mixed case, bad checksum, and invalid program length are rejected.
C-04	POSITIVE	Public certificate omits exact secrets	The returned public certificate contained none of the exact test mnemonic, seed, private scalar, or chain-code values.
C-05	POSITIVE	Runtime stayed local in the probe	Monkeypatched socket, process, URL, and write-like file APIs were not called by the reference runtime.

Important limit

The tests show conformance for the covered vectors and branches. They do not prove every BIP edge case, constant-time behavior, secure key custody, authorization, or safe deployment. Coverage was 86%, not 100%, and a test cannot validate a control it never exercises.

ELI5

The calculator got the published homework answers right. That proves useful arithmetic. It does not prove the calculator is a safe vault for the master key.

5. Main finding: the gate is a sign, not a lock

ID	Risk	Finding	Why it matters
F-01	HIGH	Authorization can be bypassed by design	derive_bip84_public_certificate accepts a mnemonic, path and passphrase but no authorization object. It directly derives the seed and private HD path.
F-02	MEDIUM	Ownership is self-asserted	authorize_audit accepts the self-owned known-seed scope without technical proof that the subject or secret belongs to the caller.
F-03	LOW	Exports enlarge the misuse surface	The package root publicly exports mnemonic_to_seed and master_node_from_seed. This is normal for a library, but it contradicts treating a separate policy function as mandatory enforcement.

Demonstrated bypass

Forbidden scope rejection worked: True. But direct derivation without an authorization certificate also worked: True. No secret search was performed; the official public test mnemonic was used.

Risk interpretation

- Exploit difficulty: trivial for code already able to import the package.
- Standalone wallet-theft impact: low, because the caller must already supply a mnemonic and the package has no search, signing, network, or broadcast function.
- Assurance impact: high if documentation claims every cryptographic operation is gated.
- Correct design choice: either describe authorize_audit honestly as an advisory policy helper, or put secret operations behind a separately authenticated and authorized service boundary.

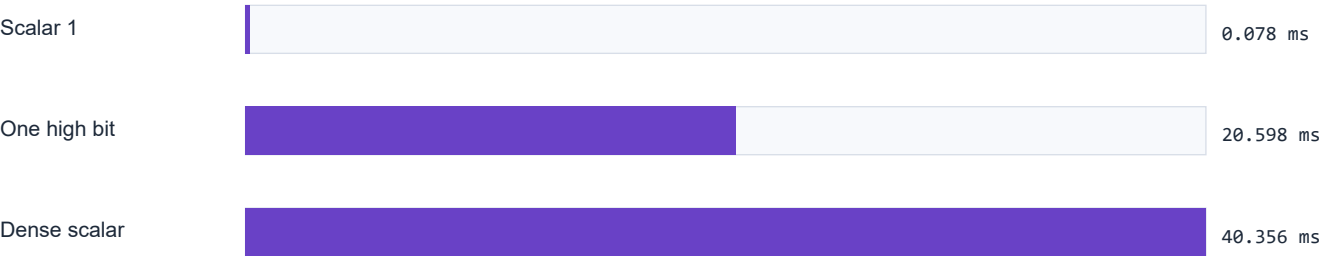
Evidence locations

src/ugts36/sara363.py:498-527, 559-583; src/ugts36/__init__.py:123-166; output/security_audit/security_probe_results.json

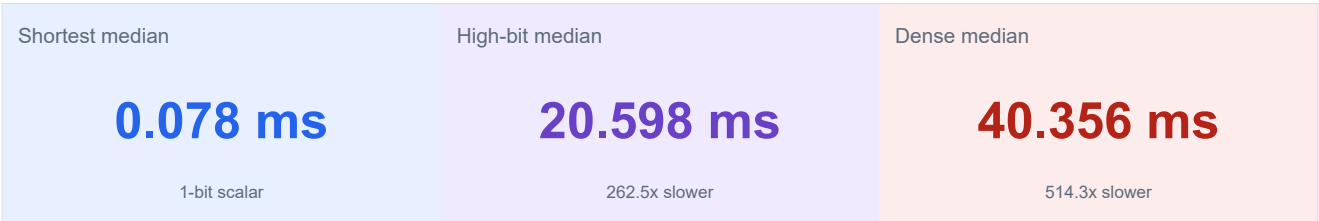
6. Secret handling and timing

ID	Risk	Finding	Why it matters
F-04	HIGH	HDNode is easy to log by accident	Its default repr and dataclasses.asdict include the private scalar and chain code. A debug print, exception context, or generic serializer could expose full wallet authority.
F-05	MEDIUM	Secrets cannot be reliably wiped	Mnemonic and passphrase are immutable Python strings; seeds and chain codes are immutable bytes; there is no zeroize method.
F-06	MEDIUM	Scalar multiplication is variable-time	The double-and-add loop branches on secret bits. Chosen-scalar measurements ranged from 0.078 ms to 40.356 ms per call, over 500x.

Chosen-scalar public-key timing (median milliseconds per call)



A production-grade secret-key implementation should not expose this input-dependent pattern.



What this does and does not prove

It proves strong input-dependent timing in this implementation. It does not prove a remote key-recovery exploit. A practical side-channel attack would require real secrets, repeated observations, and a capable local or co-resident attacker. Nevertheless, this is a production blocker: use an audited constant-time secp256k1 implementation or hardware wallet for real keys.

7. Can it hack a wallet? Feasibility by vector

PBKDF2 rate	Full BIP84 rate	Random 12 words	1M passphrases
1115/s	3.63/s	1.49e+30 y	38.3 h
seed step only	pure Python	expected at measured rate	expected half-dictionary

Question	Verdict	Plain-English meaning
Reverse a public address to a private key	INFEASIBLE	No shortcut or capability found. Public decoding reveals only witness and script structure.
Search a uniform 12-word BIP39 mnemonic	INFEASIBLE	Expected 2^{127} full trials. The package estimates work but has no enumerator.
Guess a weak passphrase after mnemonic theft	FEASIBLE	BIP39 makes every passphrase a valid seed. A known public address lets an attacker test candidates for a match.
Use a human-made or low-entropy mnemonic	DANGEROUS	Checksum validity proves syntax, not randomness or ownership. BIP39 explicitly targets computer-generated entropy.
Leak key objects through logs or dumps	FEASIBLE	Default repr/asdict expose sensitive fields, and immutable objects can linger in memory.
Bypass authorize_audit locally	TRIVIAL	Call the exported derivation API directly. This does not magically provide a missing mnemonic.
Steal funds using this package alone	NO	There is no transaction signing, broadcast, network, balance, or funds-transfer capability.

The realistic attack path

The cryptography is not the easy door. The realistic doors are a stolen mnemonic, a weak passphrase, logging, memory dumps, malware, phishing, or a tampered package. The package is a calculator that could help verify a guessed secret after the attacker already has a major piece of it; it is not a practical 128-bit brute-force engine.

8. The supplied validator can report a false sense of safety

ID	Risk	Finding	Why it matters
F-07	HIGH	Stored transcript is trusted as a test run	The validator reads test_results_3_6_3.txt and checks that it ends in OK. It does not execute the tests.
F-08	HIGH	Capability flags are self-reported	Network, transaction, and egress values are read from the stored certificate instead of observed from runtime behavior.
F-09	MEDIUM	AST scan is narrow and bypassable	Only sara363.py and sara_runtime.py are scanned. A synthetic <code>__import__('socket')</code> inside a harmless function evaded both checks.
F-10	MEDIUM	Visual inspection count is hardcoded	The report field <code>rendered_pages_visually_inspected</code> is always set to 18 and then validated as equal to 18.

ELI5

The validator asks the report card whether the student passed, rather than watching the exam. It also checks that a note says 'no network' instead of watching the program's network behavior.

Why the pristine validator still matters

It successfully verifies useful static facts: schema validity, content hashes inside the example, operator and claim counts, wordlist fingerprint, public fixture values, and PDF openability. Its final `valid=true` result must be described as a package-consistency check, not a security certification.

Evidence locations

`tools/validate_sara363_package.py:52, 69-92, 115-168; output/security_audit/validator_pristine_console.txt; output/security_audit/security_probe_results.json`

9. Integrity and supply-chain findings

Pristine ZIP	Current copy	Unexpected keys	Wheel size
152/152	150/152	0	49,610 B
manifest hashes match	two policy docs differ	pristine secret scan	code-only artifact

ID	Risk	Finding	Why it matters
F-11	MEDIUM	Current README and SECURITY hashes fail	Both differ from the pristine ZIP. Removed material includes the named non-negotiable boundary and rejected-capabilities language.
F-12	MEDIUM	Manifest is unsigned and not enforced	A writer who can replace code can also replace its hash list. Nothing automatically verifies it before import or execution.
F-13	LOW	Wheel omits project data and audit assets	The wheel contains modules and metadata only. The wordlist, schemas, examples, validator, tests, and evidence PDF are not installed.
F-14	LOW	Test and validator dependencies are undeclared	jsonschema and PyMuPDF are necessary for advertised source-tree verification but absent from pyproject metadata.

Permission check

The inspected files inherit Windows ACLs. Tom, SYSTEM, and Administrators have full control; CodexSandboxUsers has read and execute. No Everyone or broad writable Users entry appeared. These are ordinary source-code permissions, not hardened secret-storage permissions.

Secret scan

The pristine scan covered 111 UTF-8 text files and 959,852 bytes. It found no encoded xprv/WIF value and no suspicious non-empty wallet-secret JSON field. The two mnemonic occurrences are the official public BIP39 vector. Pattern scanning cannot prove absence of encrypted or obfuscated secrets.

10. Input resilience and local denial of service

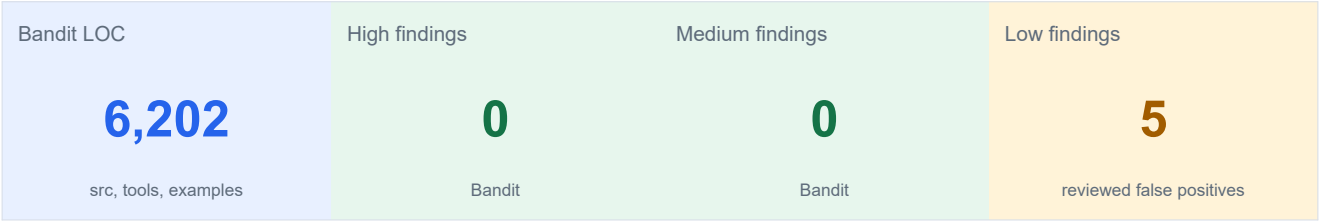
Random Bech32	Bech32 p95	Path components	Trace reuse
10,000	4.9 us	100,000	6 -> 12
zero accepted	max 20.2 us	accepted without limit	entries accumulate

ID	Risk	Finding	Why it matters
F-15	LOW	Derivation paths have no depth limit	Parsing 100,000 components took only 0.0146 s, but actually deriving them would consume substantial CPU. BIP32 serialized depth is one byte, so a practical cap is expected.
F-16	LOW	Search estimator has no bits upper bound	A one-million-bit request raised OverflowError. If exposed to untrusted input, extreme integers can become a local resource-abuse vector.
F-17	LOW	Runtime trace grows across repeated runs	run_reference appends six more entries every time because the trace is never reset.
F-18	LOW	Trace discloses an absolute local path	The wordlist path reveals the local username and package location. This is minor information leakage.

Positive parser result

The random Bech32 probe found no accidental acceptance and no latency spike. This is a useful smoke test, not exhaustive fuzzing or a proof that every parser edge case is safe.

11. Automated scanner and dependency results



Question	Verdict	Plain-English meaning
Bandit static scan	CLEAN WITH LIMITS	Five low-severity, medium-confidence items. They are test vectors, an empty optional passphrase, boolean literals, or non-secret path strings.
pip-audit isolated environment	0 KNOWN VULNS	42 installed packages examined; 0 known vulnerabilities returned.
Local project lookup	SKIPPED	ugts-kc-363-sara is local and not found on PyPI, so registry vulnerability matching cannot assess its source.
pip check	PASS	No broken installed requirements in the final isolated environment.
Wheel build	PASS	ugts_kc_363_sara-3.6.3-py3-none-any.whl built; SHA-256 4ded3bef063177ae59ddda87a8ad1fea3af13693dd19362b267a44a94e8df677.

Do not overread green scanners

Bandit did not detect the advisory authorization boundary, loggable HDNode, weak validator, or policy-document tampering. Automated scanners are one layer of evidence, not the verdict.

12. Remediation roadmap

Priority 0 - before any real mnemonic or passphrase

ID	Risk	Finding	Why it matters
R-01	HIGH	Do not market the current gate as enforced	Either rename it as an advisory policy check or place all secret operations behind a separately authenticated service/API that requires an authorized context.
R-02	HIGH	Replace Python secret-key math	Use an audited constant-time secp256k1 library or hardware wallet. Keep this code for public vectors and education only.
R-03	HIGH	Make secret objects non-printable	Set repr=False on secret fields, prevent generic dataclass serialization, minimize lifetime, avoid immutable strings where possible, and prohibit plaintext logging.
R-04	HIGH	Restore and verify policy documents	Reconcile README.md and SECURITY.md with the pristine ZIP, review the changes, and require signed manifest verification before execution.

Priority 1 - make verification meaningful

- Make the validator run tests itself, capture the exit code and fresh timestamp, and reject stale transcripts.
- Observe runtime I/O with an allowlist or sandbox instead of trusting stored capability booleans.
- Scan the complete installed artifact and dynamic-import patterns; verify the manifest and signature first.
- Remove hardcoded visual-inspection claims; generate page renders and record verifiable inspection evidence.
- Add explicit developer/test dependency metadata with compatible pins or hashes.

Priority 2 - resilience and privacy

- Limit derivation depth to the supported BIP32 range and cap input length before parsing.
- Limit search-estimator bits to a documented range, such as 1-256, and return a controlled validation error.
- Reset or bound runtime traces and store only a basename or logical wordlist identifier.
- Decide whether the wheel is code-only or self-contained; package data accordingly and test the installed wheel.

13. Evidence map and coverage limits

All audit evidence is under `output/security_audit` in the package root. The report generator reads the machine-readable JSON files directly.

Reference	URL
Final 195-test run	<code>output/security_audit/unittest_final_console.txt</code>
Branch coverage	<code>output/security_audit/coverage_report.txt</code> and <code>coverage.json</code>
Adversarial probes	<code>output/security_audit/security_probe_results.json</code>
Timing evidence	<code>output/security_audit/side_channel_timing.json</code>
Pristine/current integrity	<code>output/security_audit/manifest_zip.json</code> and <code>manifest_filesystem.json</code>
Policy document comparison	<code>output/security_audit/documentation_integrity.json</code>
Packaged-secret scan	<code>output/security_audit/packaged_secret_scan.json</code>
Static and dependency scans	<code>output/security_audit/bandit.json</code> and <code>pip_audit_isolated.json</code>
Validator run	<code>output/security_audit/validator_pristine_console.txt</code>
Wheel inspection	<code>output/security_audit/wheel_inspection_console.txt</code>
Windows ACLs	<code>output/security_audit/windows_acl_report.txt</code>
Machine profile	<code>output/security_audit/environment.txt</code>

What remains unproven

- No exhaustive proof of BIP39/BIP32/BIP84/Bech32 correctness beyond covered vectors and branches.
- No cryptographic-library certification, secure-code review by multiple independent reviewers, or formal verification.
- No practical side-channel key extraction; only a timing-pattern demonstration.
- No malware, process-memory dump, backup, browser, clipboard, or user-operational assessment.
- No external wallet ownership, balance, or on-chain activity verification.
- No offensive testing against any third-party system or secret.

Confidence statement

High confidence in the recorded local observations and covered public vectors. Medium confidence in broader parser and packaging conclusions. Low confidence for any claim of production wallet safety, because the design intentionally lacks hardened secret custody and constant-time primitives.

14. Authoritative references

These standards and guidance were checked on 18 August 2026. Local package claims were judged against primary Bitcoin BIP specifications and current packaging/security guidance.

Reference	URL
S1 - BIP39 mnemonic and PBKDF2	https://github.com/bitcoin/bips/blob/master/bip-0039.mediawiki
S2 - BIP32 hierarchical deterministic wallets	https://github.com/bitcoin/bips/blob/master/bip-0032.mediawiki
S3 - BIP84 P2WPKH derivation	https://github.com/bitcoin/bips/blob/master/bip-0084.mediawiki
S4 - BIP173 Bech32	https://github.com/bitcoin/bips/blob/master/bip-0173.mediawiki
S5 - BIP350 Bech32m	https://github.com/bitcoin/bips/blob/master/bip-0350.mediawiki
S6 - OWASP secrets management	https://cheatsheetseries.owasp.org/cheatsheets/Secrets_Management_Cheat_Sheet.html
S7 - Python packaging and package data	https://packaging.python.org/en/latest/guides/distributing-packages-using-setuptools/

Key standard facts used

- BIP39 specifies 128-256 bits of computer-generated entropy, NFKD normalization, and PBKDF2-HMAC-SHA512 with 2,048 iterations and a 64-byte output.
- BIP39 says the scheme is for transporting computer-generated randomness, not for user-created brainwallet sentences.
- BIP84 specifies `m / purpose' / coin_type' / account' / change / address_index` with purpose 84' and publishes the public vectors used here.
- OWASP recommends minimizing plaintext secret lifetime, avoiding immutable secret storage where possible, zeroing memory, applying least privilege, and never logging plaintext secrets.
- Python packaging guidance requires dependencies and package data to be declared when they are needed by an installed distribution.

Final verdict

Conditionally feasible as a public standards reference. Not approved for real wallet secrets.

The package's covered Bitcoin arithmetic and public certificate pipeline work. The pristine archive is internally consistent, and the runtime showed no network, process, or write behavior in the probe. However, authorization is not enforced, secret-bearing objects are easy to serialize, elliptic-curve math is variable-time, the validator can produce false confidence, and the current extracted policy documents fail their original hashes.

Answer to the practical question

No: this is not a serious full-wallet hacking engine. A public address alone does not make a recoverable private key. Uniform 12-word brute force is astronomically infeasible. Yes: weak operational secrets are still dangerous. If a mnemonic is already stolen, a weak BIP39 passphrase can be tested; accidental repr/asdict logging can reveal key material; and the advisory gate can be ignored by local callers.

Use for education	Use for public decode	Use for real secrets	Treat validator as cert
YES	YES	NO	NO
public vectors only	watch-only facts	until P0 fixes	consistency check only

Report produced from the captured evidence listed on page 13. This is a technical assessment, not a guarantee, financial advice, or authorization to test third-party systems.